

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: PF	Course Code: CMPE-112L
Assignment Type: Lab	Dated: 04-05-2026
Semester: 1	Session: Spring 2026
Lab/Project/Assignment #: 1	CLOs to be covered: CLO 3
Lab Title: Sets,dictionary	Teacher Name: Hafiz Muhammad Mubashar

Lab Evaluation:

CLO1	Apply appropriate programming techniques to create executable programs to solve well defined problems					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 8

Lab Goals/Objectives:

- To understand the concept of Python sets.
- To perform different set operations such as union, intersection, difference, and symmetric difference.
- To learn the use of built-in set methods like `add()`, `remove()`, `discard()`, `pop()`, `clear()`, and `update()`.
- To develop problem-solving skills using Python sets.

Equipment Required:

- Laptop/PC
- Python 3.x
- VS Code (or any Python IDE)
- Windows Operating System

Theory:

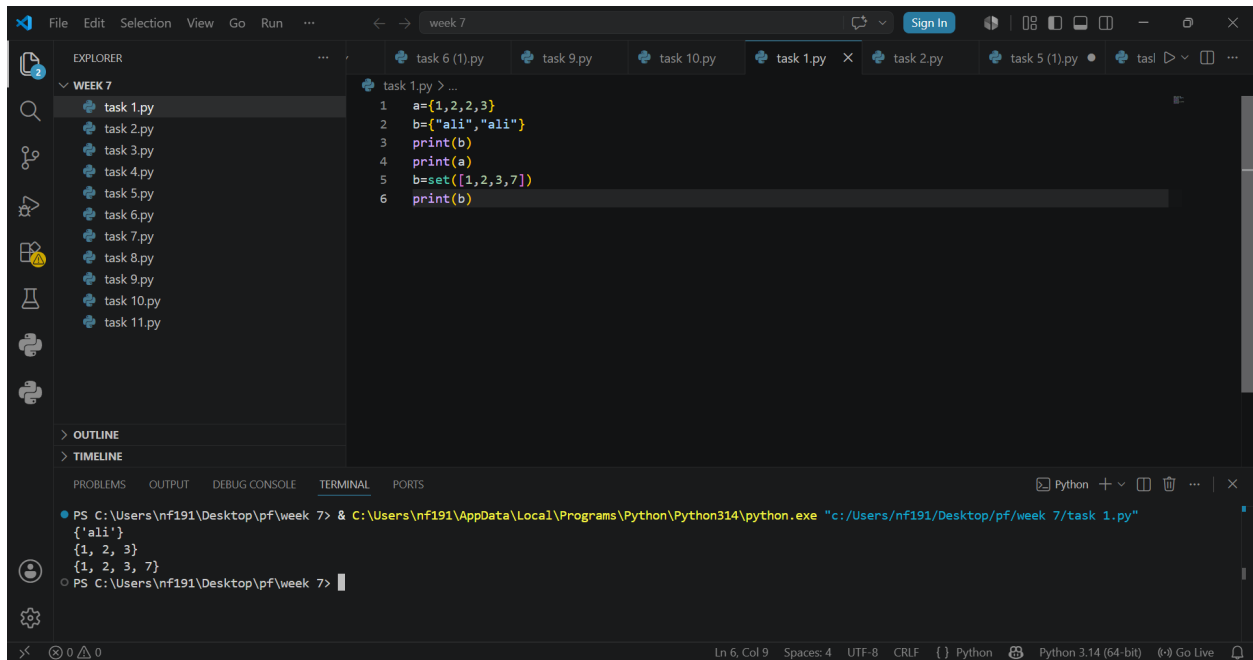
A **set** is an unordered collection of unique elements in Python. Sets do not allow duplicate values and are mutable, meaning elements can be added or removed. Python provides various built-in methods and operations to manipulate sets, such as union, intersection, difference, symmetric difference, and membership testing. Sets are commonly used for storing distinct values and performing mathematical set operations efficiently.

Lab Work:

Task 1:

Write a program to create sets and demonstrate that duplicate values are automatically removed from a set. Also create a set using the `set()` constructor and display the output

Result:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'WEEK 7' with files task 1.py through task 11.py. The main editor displays task 1.py with the following code:

```
1 a={1,2,2,3}
2 b={"ali","ali"}
3 print(b)
4 print(a)
5 b=set([1,2,3,7])
6 print(b)
```

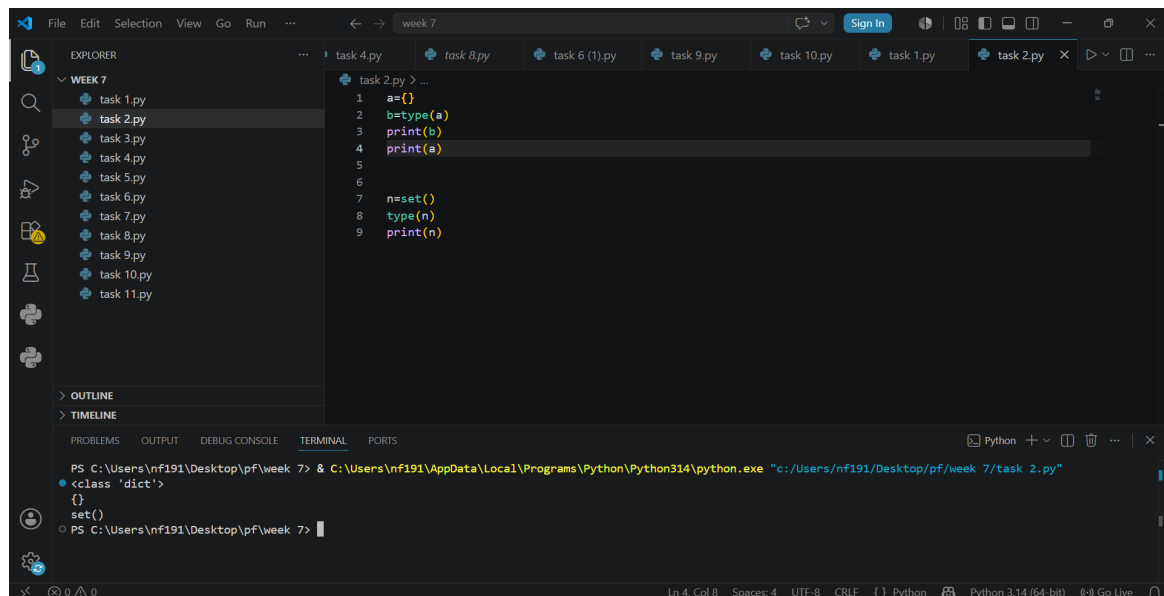
The TERMINAL pane at the bottom shows the command executed: `PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 1.py"`. The output is:

```
{'ali', 'ali'}
{1, 2, 3}
{1, 2, 3, 7}
```

Task 2:

Write a program to demonstrate the difference between an empty dictionary `{}` and an empty set created using `set()`. Display the data types and output.

Results:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows a project named 'WEEK 7' with files task 1.py through task 11.py. The main editor displays task 2.py with the following code:

```
1 a={}
2 b=type(a)
3 print(b)
4 print(a)
5
6
7 n=set()
8 type(n)
9 print(n)
```

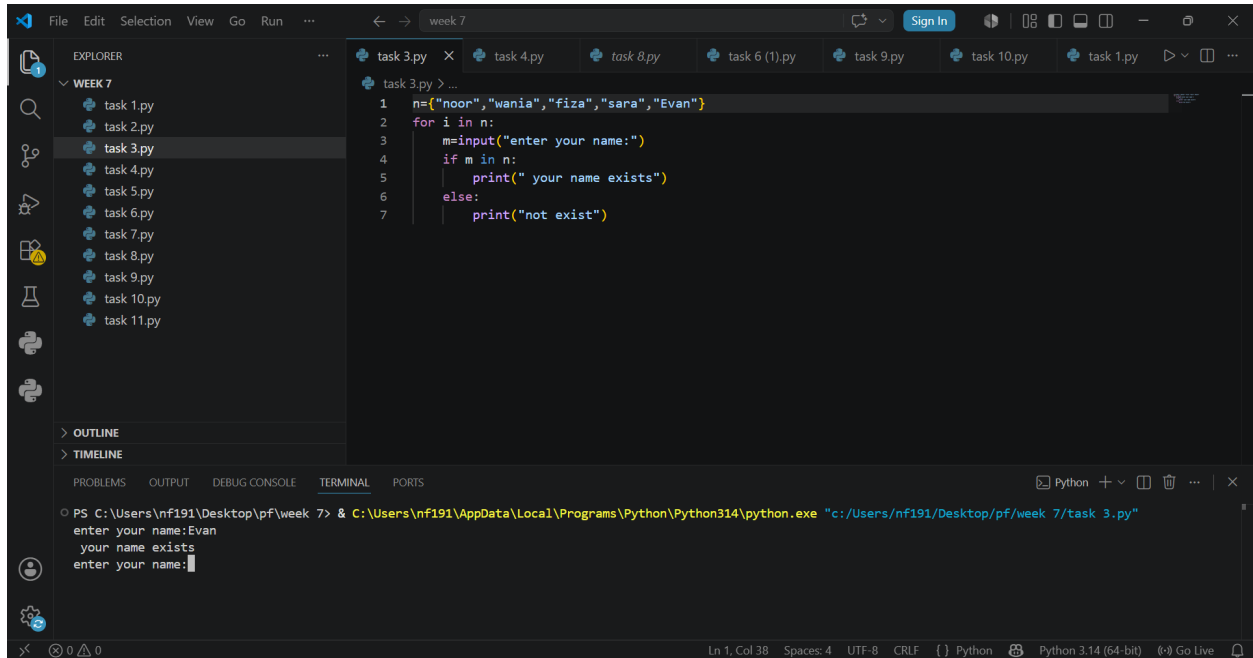
The TERMINAL pane at the bottom shows the command executed: `PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 2.py"`. The output is:

```
<class 'dict'>
{}
set()
```

Task 3:

Write a program to create a set of names and check whether a user-entered name exists in the set using the membership operator.

Result:



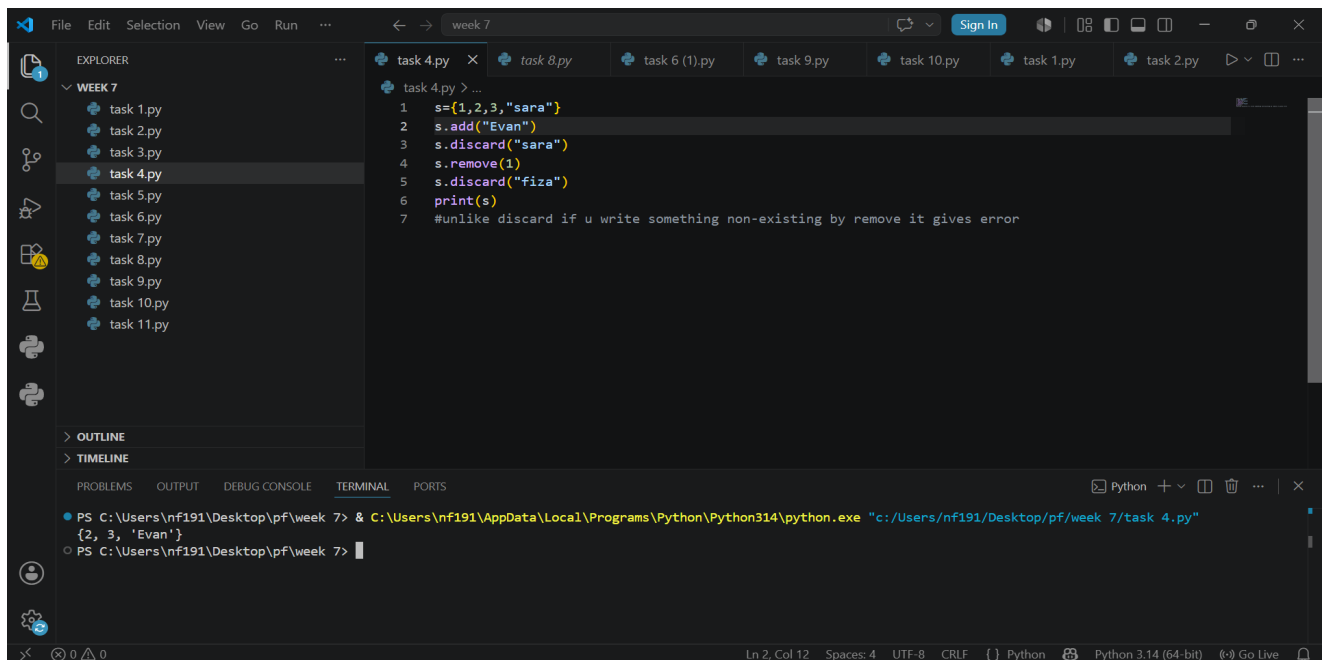
```
task 3.py > ...
1 n={"noon","wania","fiza","sara","Evan"}
2 for i in n:
3     m=input("enter your name:")
4     if m in n:
5         print(" your name exists")
6     else:
7         print("not exist")
```

```
PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 3.py"
enter your name:Evan
your name exists
enter your name:
```

Task 4:

Write a program to perform basic set operations including add(), remove(), and discard(), and observe their behavior.

Result:



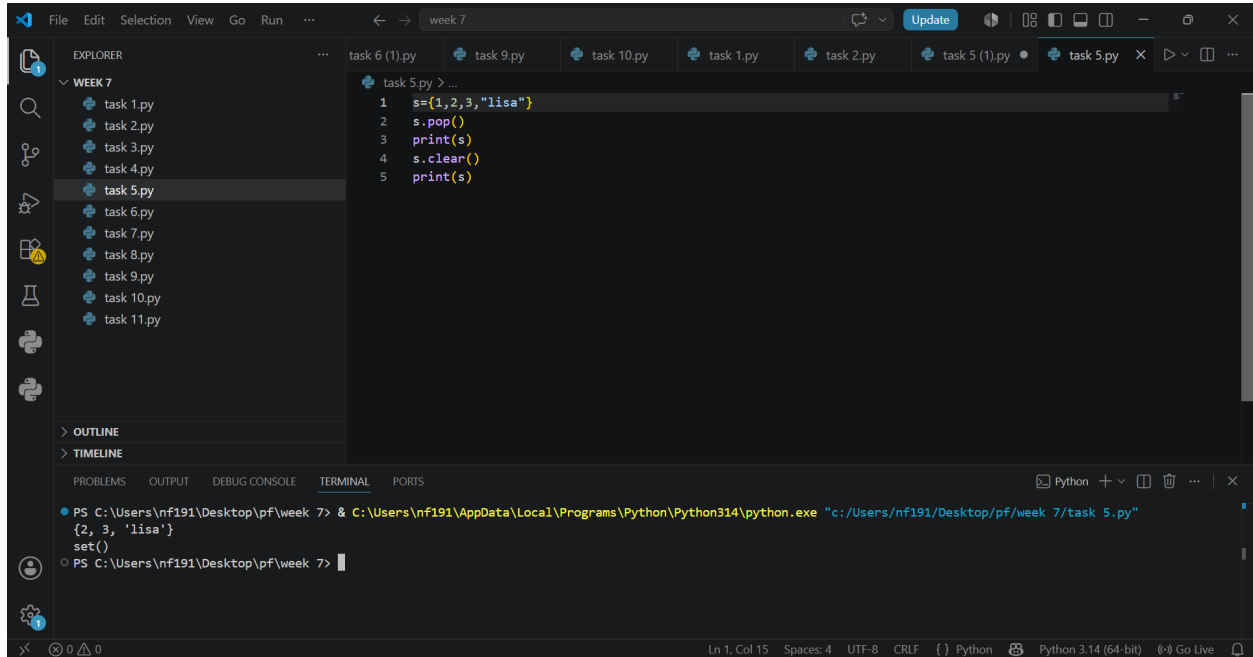
```
task 4.py > ...
1 s={1,2,3,"sara"}
2 s.add("Evan")
3 s.discard("sara")
4 s.remove(1)
5 s.discard("fiza")
6 print(s)
7 #unlike discard if u write something non-existing by remove it gives error
```

```
PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 4.py"
{2, 3, 'Evan'}
```

Task 5:

Write a program to demonstrate the use of the `pop()` and `clear()` methods on a set.

Result:



The screenshot shows the Visual Studio Code editor with a file explorer on the left listing files from 'task 1.py' to 'task 11.py'. The main editor window displays 'task 5.py' with the following Python code:

```
1 s={1,2,3,"lisa"}
2 s.pop()
3 print(s)
4 s.clear()
5 print(s)
```

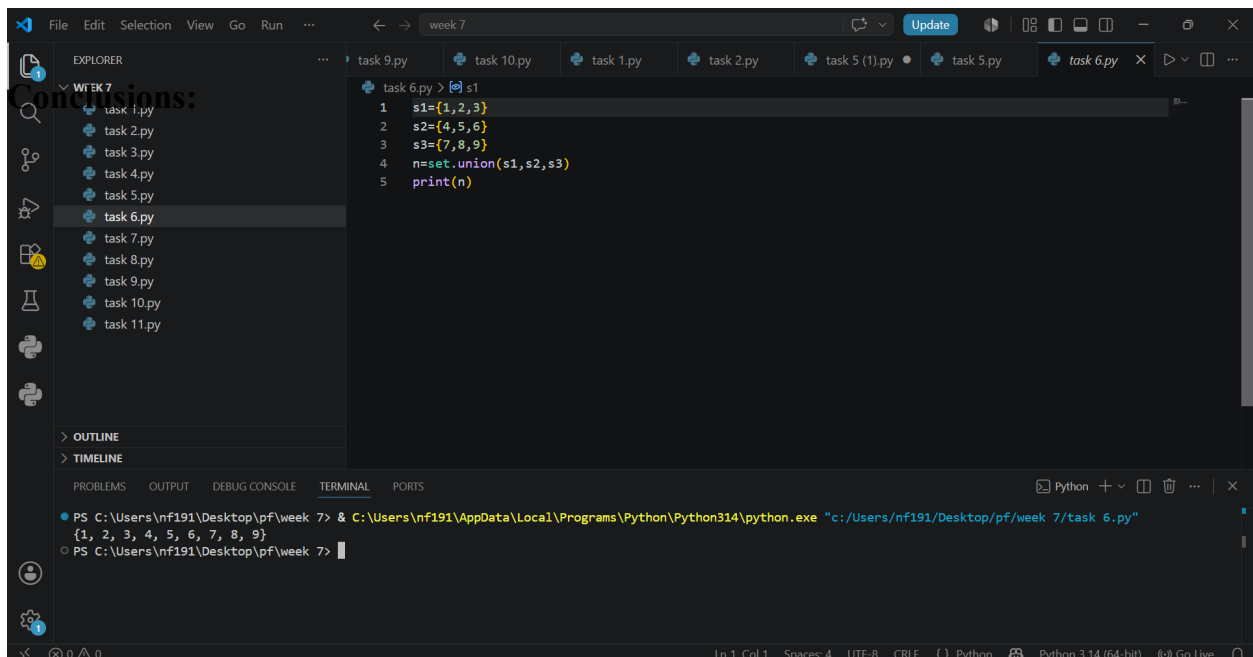
The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 5.py"
{2, 3, 'lisa'}
set()
PS C:\Users\nf191\Desktop\pf\week 7>
```

Task 6:

Write a program to find the union of three sets using the `union()` method.

Result:



The screenshot shows the Visual Studio Code editor with a file explorer on the left listing files from 'task 1.py' to 'task 11.py'. The main editor window displays 'task 6.py' with the following Python code:

```
1 s1={1,2,3}
2 s2={4,5,6}
3 s3={7,8,9}
4 n=set.union(s1,s2,s3)
5 print(n)
```

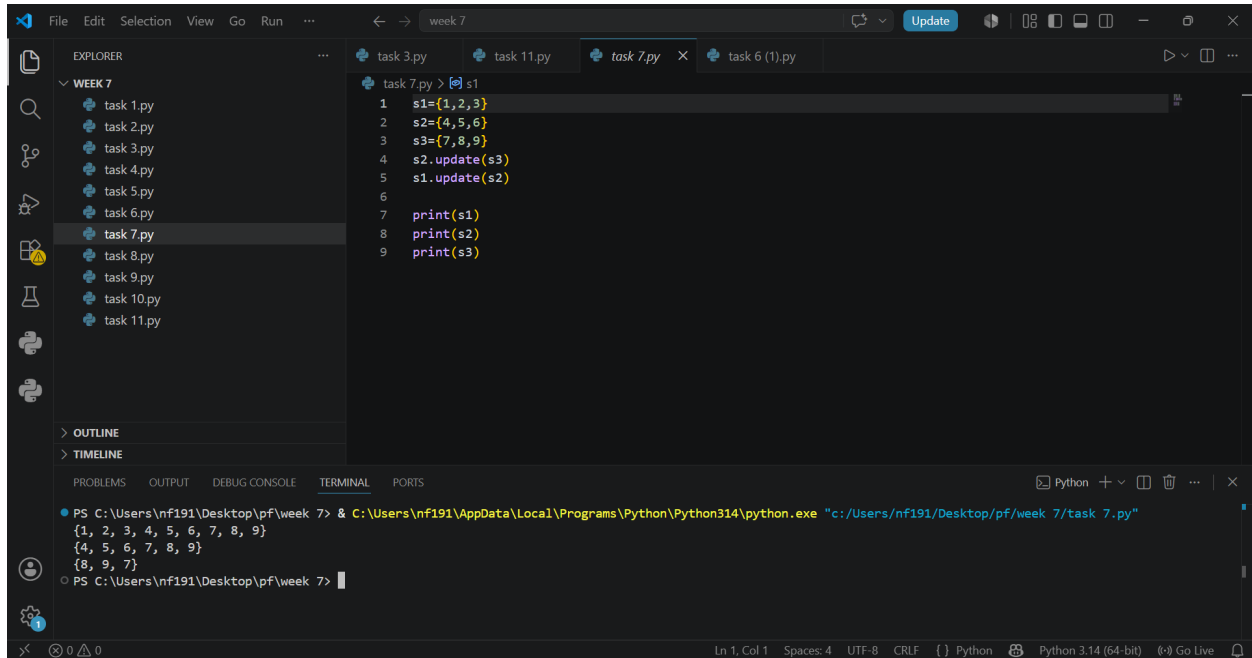
The terminal at the bottom shows the command to run the script and its output:

```
PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 6.py"
{1, 2, 3, 4, 5, 6, 7, 8, 9}
PS C:\Users\nf191\Desktop\pf\week 7>
```

Task 7:

Write a program to update one set with the elements of other sets using the `update()` method and display the results.

Result:



The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files from 'WEEK 7' (task 1.py to task 11.py). The main editor displays 'task 7.py' with the following Python code:

```
1 s1={1,2,3}
2 s2={4,5,6}
3 s3={7,8,9}
4 s2.update(s3)
5 s1.update(s2)
6
7 print(s1)
8 print(s2)
9 print(s3)
```

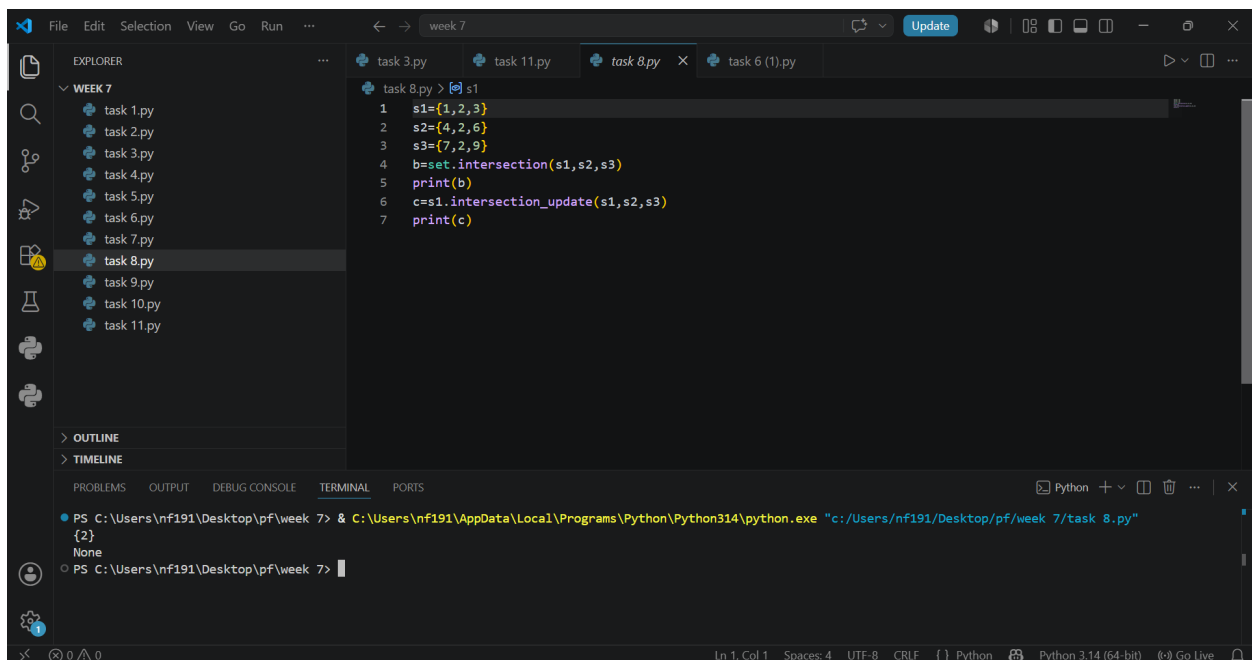
The bottom terminal panel shows the command `python.exe "c:/Users/nf191/Desktop/pf/week 7/task 7.py"` and its output:

```
{1, 2, 3, 4, 5, 6, 7, 8, 9}
{4, 5, 6, 7, 8, 9}
{8, 9, 7}
```

Task 8:

Write a program to find the intersection of multiple sets using the `intersection()` and `intersection_update()` methods.

Result:



The screenshot shows the Visual Studio Code editor with a file explorer on the left containing files from 'WEEK 7' (task 1.py to task 11.py). The main editor displays 'task 8.py' with the following Python code:

```
1 s1={1,2,3}
2 s2={4,2,6}
3 s3={7,2,9}
4 b=set.intersection(s1,s2,s3)
5 print(b)
6 c=s1.intersection_update(s1,s2,s3)
7 print(c)
```

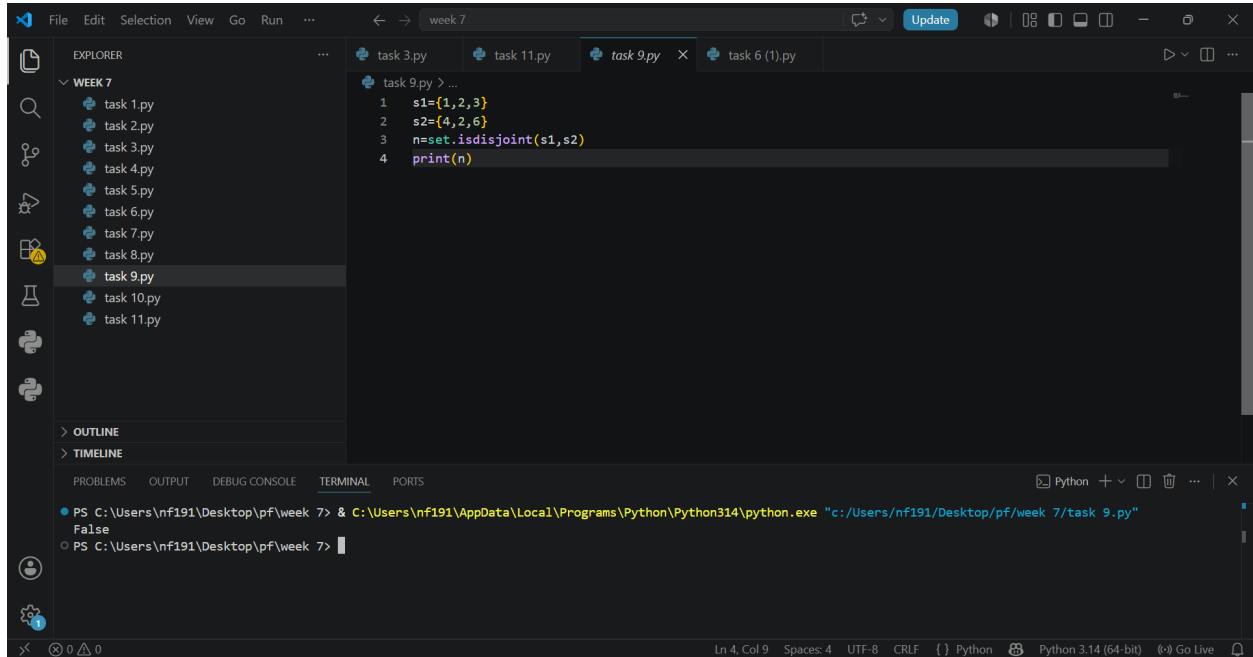
The bottom terminal panel shows the command `python.exe "c:/Users/nf191/Desktop/pf/week 7/task 8.py"` and its output:

```
{2}
None
```

Task 9:

Write a program to check whether two sets are disjoint using the `isdisjoint()` method.

Result:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files for 'WEEK 7', with 'task 9.py' selected. The main editor displays the code for 'task 9.py':

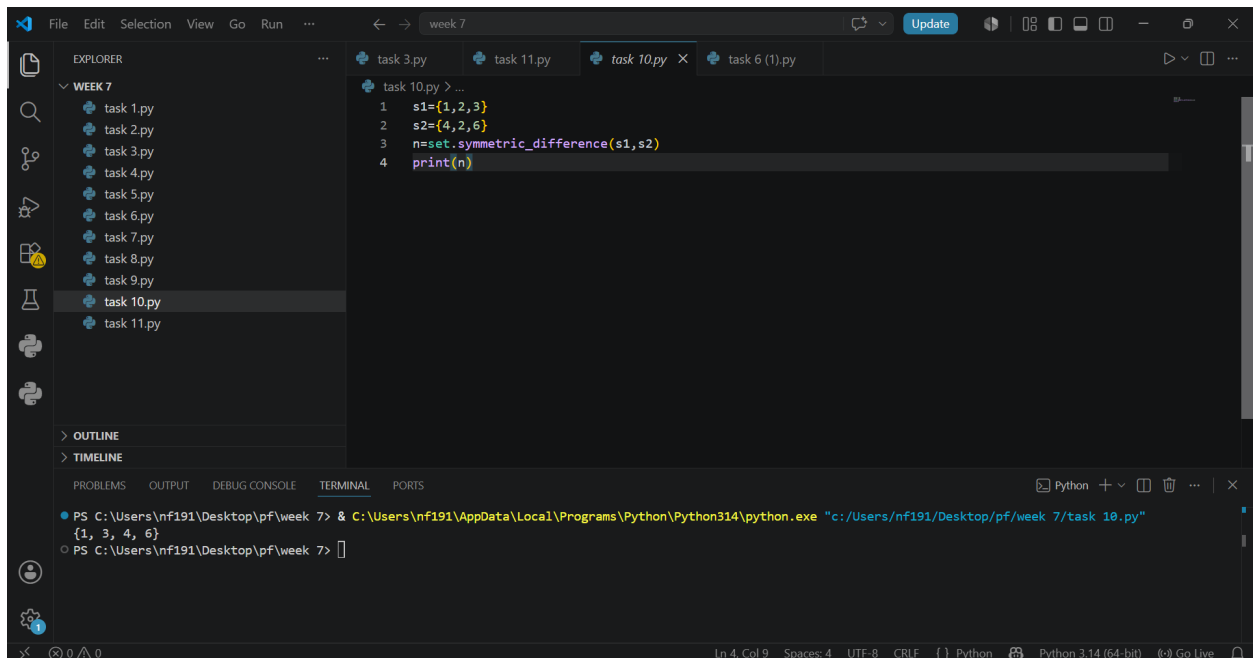
```
1 s1={1,2,3}
2 s2={4,2,6}
3 n=set.isdisjoint(s1,s2)
4 print(n)
```

The TERMINAL pane at the bottom shows the command executed: `C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 9.py"`, with the output `False`.

Task 10:

Write a program to find the symmetric difference between two sets using the `symmetric_difference()` method.

Result:



The screenshot shows the Visual Studio Code interface. The Explorer pane on the left lists files for 'WEEK 7', with 'task 10.py' selected. The main editor displays the code for 'task 10.py':

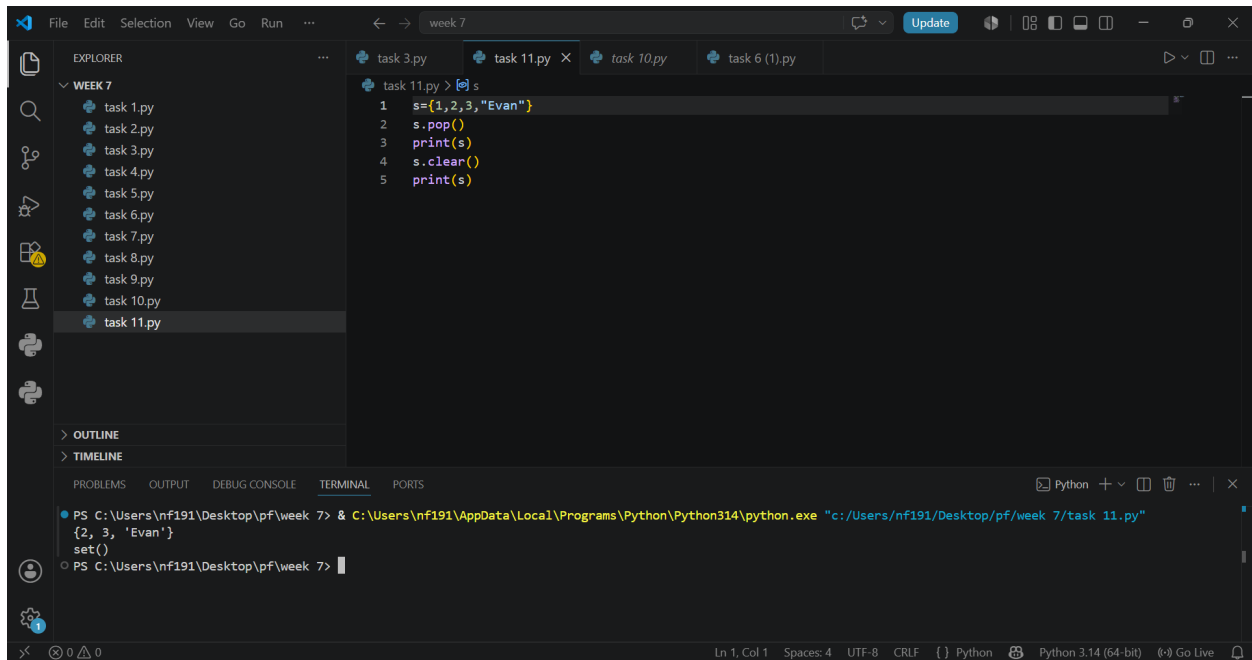
```
1 s1={1,2,3}
2 s2={4,2,6}
3 n=set.symmetric_difference(s1,s2)
4 print(n)
```

The TERMINAL pane at the bottom shows the command executed: `C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 10.py"`, with the output `{1, 3, 4, 6}`.

Task 11:

Write a program to demonstrate the use of the `pop()` and `clear()` methods on a set and display the resulting set after each operation.

Result:



The screenshot shows a Visual Studio Code editor window with a file explorer on the left displaying a directory named 'WEEK 7' containing files 'task 1.py' through 'task 11.py'. The main editor area shows the content of 'task 11.py', which contains the following Python code:

```
1 s={1,2,3,"Evan"}
2 s.pop()
3 print(s)
4 s.clear()
5 print(s)
```

Below the editor, the TERMINAL panel shows the command used to run the script: `PS C:\Users\nf191\Desktop\pf\week 7> & C:\Users\nf191\AppData\Local\Programs\Python\Python314\python.exe "c:/Users/nf191/Desktop/pf/week 7/task 11.py"`. The output of the script is displayed as:

```
{2, 3, 'Evan'}
set()
```

Conclusion:

In this lab, we learned how to create and manipulate sets in python. We performed various set operations such as union, intersection, symmetric difference, and membership testing. We also practiced using different built-in set methods including `add()`, `remove()`, `discard()`, `update()`, `pop()`, and `clear()`. This lab improved understanding of python sets and their practical applications in programming